

FinRAG NVIDIA: An Agentic Retrieval-Augmented Generation System for Financial Document Analysis

Noam Shaphir

Holon Institute of Technology

noam.shaphir@gmail.com

Abstract

Large Language Models (LLMs) struggle with precise mathematical reasoning and targeted information retrieval when applied to dense financial documents. This report presents FinRAG NVIDIA, an Agentic Retrieval-Augmented Generation (RAG) system designed to process and query corporate financial reports. By utilizing a local LLM orchestrator equipped with deterministic programmatic tools and vector similarity search, the system mitigates numeric hallucinations and ensures temporally accurate data retrieval. The architecture supports natural language queries regarding complex financial metrics, exact growth calculations, and real-time market data retrieval.

1 Introduction

Financial reports contain dense, tabular data and strict chronological structures that standard text-generation models often misinterpret or hallucinate. The primary objective of this project is to develop an Agentic RAG architecture that accurately queries NVIDIA's Fiscal Year 2026 reports.

To address the limitations of standalone LLMs specifically their inability to perform high-precision arithmetic and route queries to specific temporal documents the proposed system introduces a tool-augmented orchestration approach. The framework relies entirely on local processing for document embedding and query generation, integrating external APIs only for real-time market data and automated alerts.

2 System Architecture

2.1 Document Ingestion and Vector Retrieval

Financial reports were ingested and processed into a local Chroma vector database. To preserve the structural integrity of financial tables, text splitting was optimized using a chunk size of 1000 characters with a 150-character overlap. The retrieval

mechanism utilizes cosine similarity to match the HuggingFace sentence-transformers embedding space, isolating semantic direction and reducing document chunk length bias. Furthermore, the system implements targeted search filtering; queries explicitly referencing a specific quarter (e.g., "Q3") are programmatically routed to the corresponding document (e.g., *third_q_25.pdf*) to prevent cross-quarter data contamination.

2.2 Agentic Orchestration and Deterministic Tooling

The core orchestrator is a locally hosted Llama 3.2 model, configured via LangGraph. Instead of relying on the LLM's internal token prediction for mathematical queries, the agent accesses a dedicated programmatic float calculation tool. For example, when calculating revenue growth, the LLM extracts the numeric values and delegates the arithmetic to the CPU, guaranteeing precision. Additional tools include real-time stock price retrieval via the Yahoo Finance API and an automated SMTP alert system triggered by significant price fluctuations.

2.3 Deployment and Hardware Adaptation

To facilitate real-world usability, the agent is deployed via a FastAPI backend, enabling execution as a standalone web application alongside the standard Jupyter Notebook environment. The frontend dashboard integrates asynchronous state management to display real-time API fetches and SMTP alert dispatches. Furthermore, the system architecture includes a dynamic hardware fallback mechanism. During initialization, the orchestrator attempts to load the primary LLM into VRAM; if GPU allocation fails, it automatically defaults to CPU execution or downgrades to a lightweight 1B parameter model, ensuring robust cross-platform functionality.

3 Qualitative Evaluation

The system was evaluated across several financial analysis tasks to verify retrieval accuracy and computational correctness:

- **Temporal Accuracy:** When queried about specific quarters (e.g., Q3 2025 revenue), the routing mechanism successfully isolated the relevant document, retrieving the exact figure (\$57.01 billion) without hallucinating historical data.
- **Computational Precision:** Multi-document comparisons (e.g., Q2 vs. Q3 revenue) demonstrated the efficacy of the calculation tool. The agent correctly formulated a +21.96% growth rate using deterministic arithmetic rather than predictive text generation.
- **External Integration:** The agent successfully retrieved current stock valuations using the external API and demonstrated the ability to dispatch asynchronous alerts based on predefined fluctuation thresholds.

4 Conclusion

The FinRAG NVIDIA project illustrates the practical advantages of Agentic RAG paradigms over standard retrieval systems in the financial domain. By decoupling mathematical reasoning from the LLM and utilizing targeted temporal routing, the system provides deterministic, reliable answers for complex document analysis. Future work should focus on integrating advanced visual-language models (VLMs) or specialized table-parsing algorithms to eliminate the reliance on manual table extraction.

Limitations

While the system demonstrates high accuracy for its specific dataset, it exhibits several architectural limitations. First, extracting tabular data from PDFs remains highly inconsistent using standard recursive character splitting. Consequently, the current implementation relies on hardcoded context injection for specific core income statement tables to guarantee accurate retrieval. Second, the system is explicitly over-fitted to NVIDIA's FY2026 reporting structure and nomenclature via rigid prompt engineering; generalizing the agent to process diverse financial reports from other entities would require significant refactoring of the parsing logic

and system prompts. Third, the system lacks explicit out-of-domain guardrails. When queried about financial metrics of companies other than NVIDIA, the model frequently hallucinates data or erroneously attempts to map the existing vector context to the unrelated query, failing to cleanly decline the prompt.